

Compiler Theory

Milestone 1

- **Regular Expression for each token**
- **DFA**
- **Token Class Table**
- **Scanner Function**

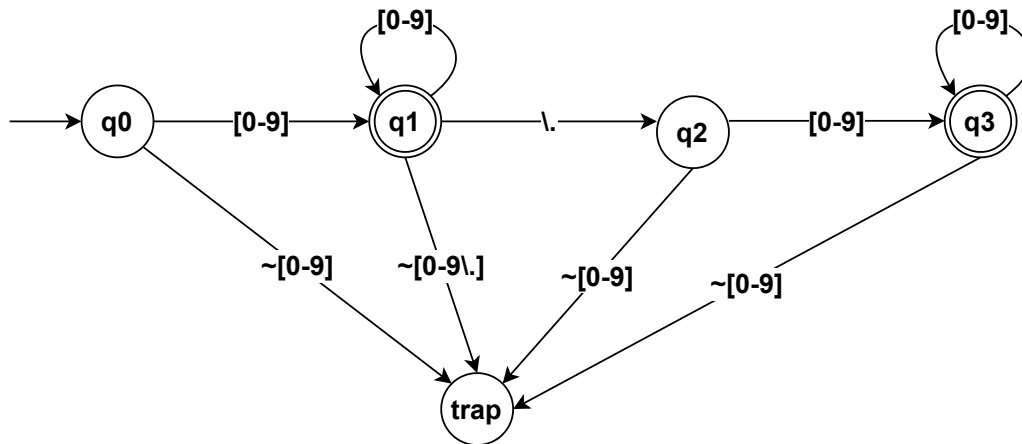
Team 41

Members:

- | | |
|---------------------------|------------|
| • Mahmoud Ahmed Attia | 2023170547 |
| • Mahmoud Ahmed Mohamed | 2023170548 |
| • Omar Ashraf Abd Rabbuh | 2023170383 |
| • Motasem Moatz | 2023170623 |
| • Youssef El Adly Mohamed | 2022170513 |
| • Youssef Hamada Emam | 2023170726 |

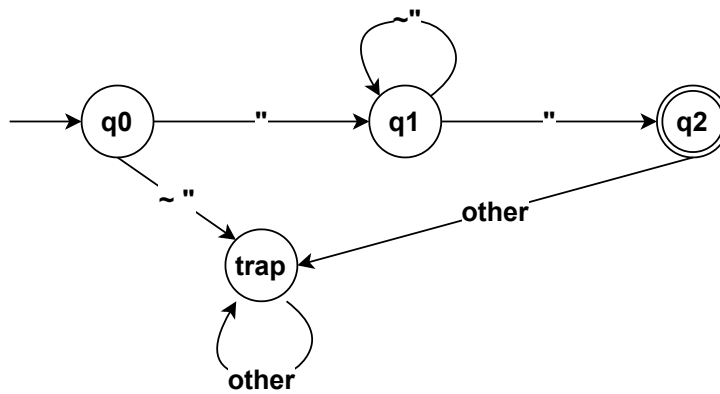
(1) Number

$[0-9]^+([0-9]^+)?$



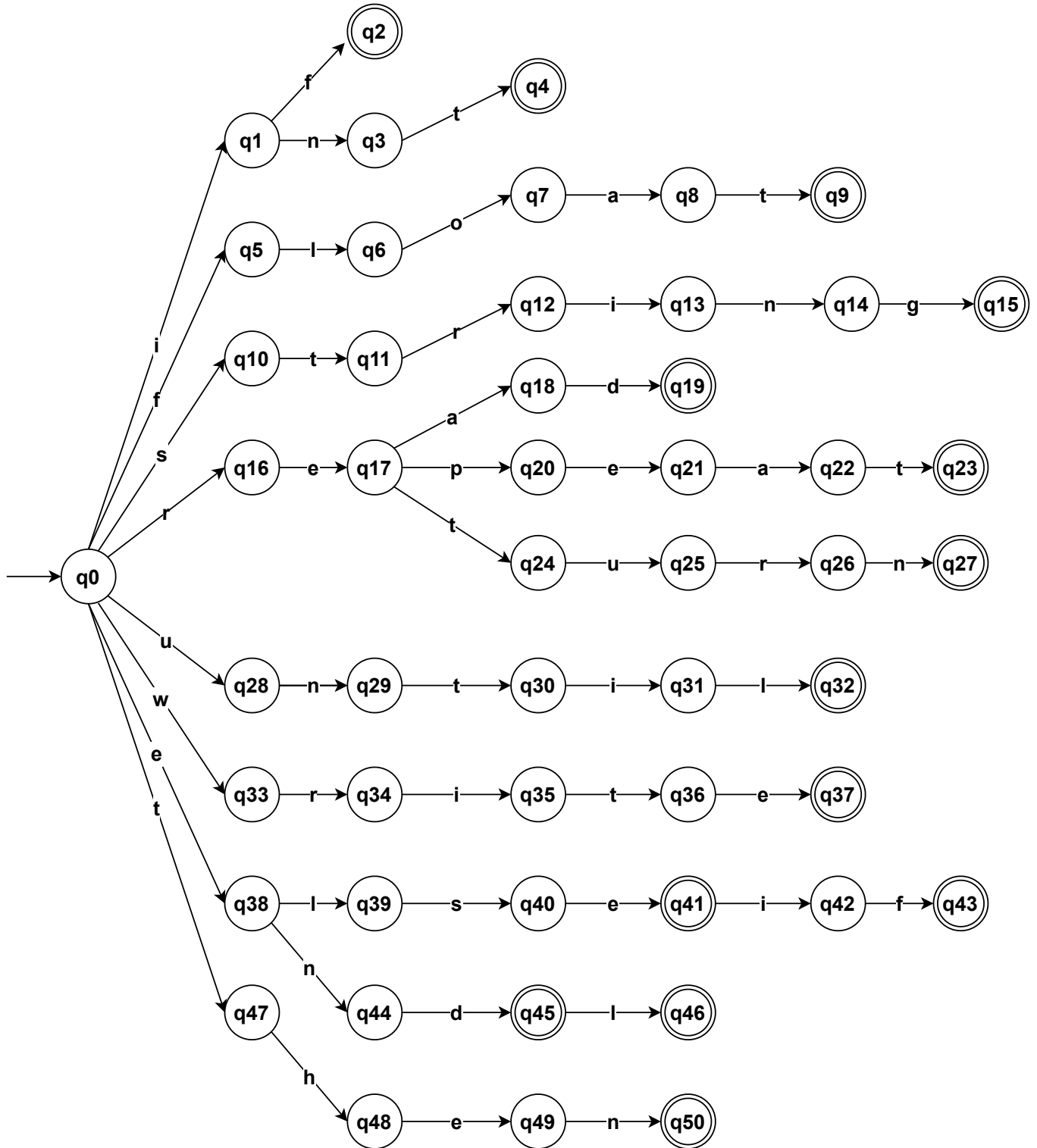
(2) String

$"(\sim)^*"$

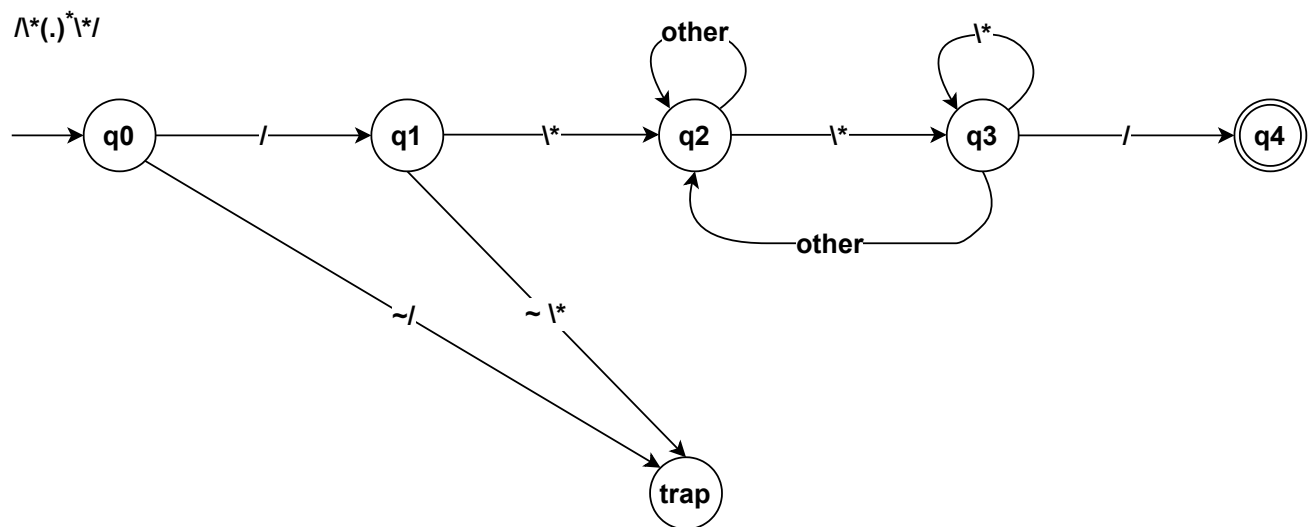


(3) Reserved Keyword

int | float | string | read | write | repeat | until | if | elseif | else | then | return | endl | end

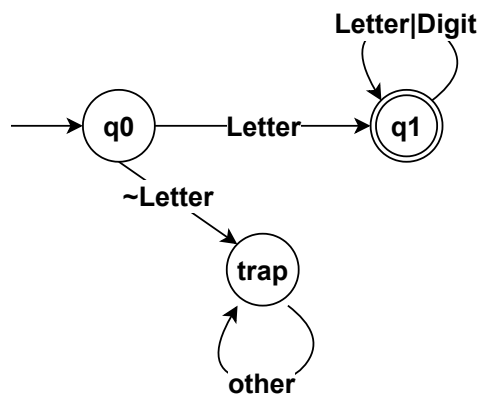


(4) Comment Statement



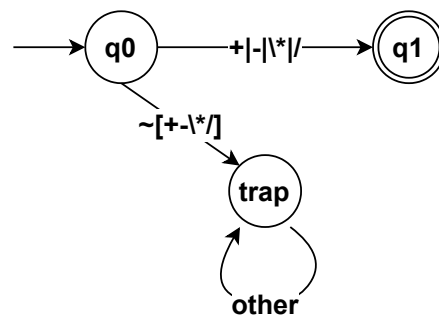
(5) Identifier

$\text{Letter}(\text{Letter}|\text{Digit})^*$



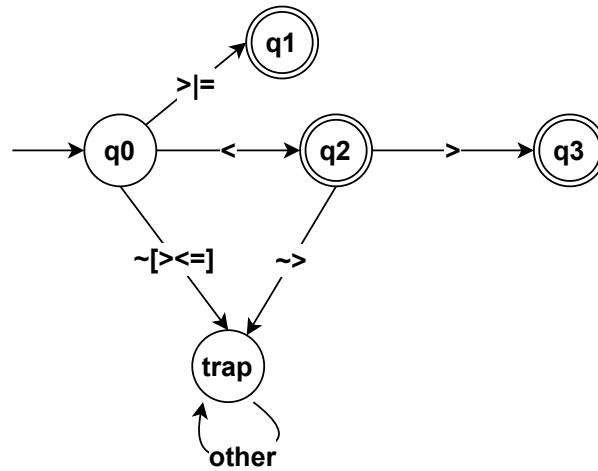
(8) Arithmetic Operator

$+|-|\backslash^*$



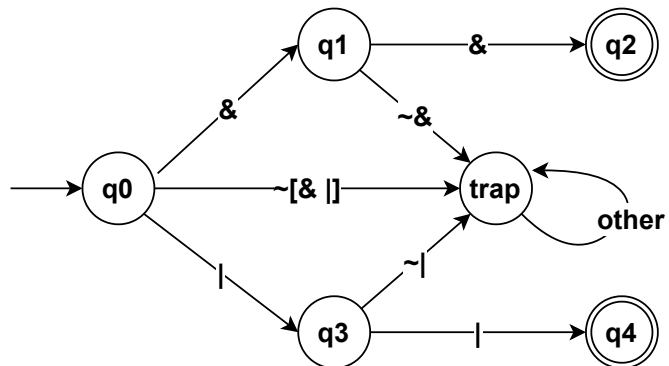
(17) Condition Operator

>|<|=|<>



(19) Boolean Operator

&& | ||



Extracted Token Class

Token Class	Regex
Int	int
Float	float
String	string
Read	read
Write	write
Repeat	repeat
Until	until
If	if
Elseif	elseif
Else	else
Then	then
Return	return
Endl	endl
End	end
Identifier	[a-zA-Z]([a-zA-Z] [0-9])*
Number	[0-9]+(\.[0-9]+)?
Comment	/*(.)**/
Tiny_String	"(^")**
Semicolon	;
Comma	,
LParanthesis	(
RParanthesis	)
EqualOp	=

Token Class	Regex
NotEqualOp	<>
LessThanOp	<
GreaterThanOp	>
AndOp	&&
OrOp	
PlusOp	+
MinusOp	-
MultiplyOp	*
DivideOp	/
AssignmentOp	:=
LCurlyBracket	{
RCurlyBracket	}

//Reserved words

Int, Float, String, Read, Write, Repeat, Until, If, Elseif, Else, Then, Return, Endl, End,

//Values

Identifier, Number, Comment, tiny_String,

//Other Chars

Semicolon, Comma, LParanthesis, RParanthesis,EqualOp, NotEqualOp, LessThanOp, GreaterThanOp, AndOp, OrOp, PlusOp, MinusOp, MultiplyOp, DivideOp, AssignmentOp,LCurlyBracket, RCurlyBracket